
MATERIAL DE APOIO – Complementar aos Slides da Aula

1. Introdução

Prezados discentes da disciplina este material tem o **objetivo geral de complementar e expandir os conhecimentos** repassados em aula, sobre os fundamentos da linguagem Python e desenvolver pequenos programas utilizando estruturas básicas da linguagem.

Aproveite para treinar.

Segue ainda, os fontes de programas em Python3, no arquivo de Jupyter Notebook.

2. Desenvolvimento

Seguem abaixo, na estrutura de Dicas, numeradas para melhor compreensão, desenvolvimento e roteiros complementares dos estudos, complementados pelos slides, webaulas, referências bibliográficas e arquivo Jupyter Notebook.

Bons estudos!

Dica 1 — O ambiente - preparação

Fundamentos de Python - Ambiente Jupyter Notebook

"""

A ferramenta estará disponível em <https://www.python.org/downloads/>, ou ainda por meio da instalação:

--> no ambiente Windows, pode-se fazer o "Download Windows installer (64-bit)" ou entra no seu CMD e digitar o comando python, que ele fará a instalação.

--> no linux é nativo. caso sua distro não tenha, para os derivados debian: sudo apt install python;

--> para os demais casos, seguir as orientações do site supramencionado.

-> python online: https://www.onlinegdb.com/online_python_compiler ou <https://pythononline.net/>

"""

print("Acima, exemplo de bloco comentado com 1 (uma) e várias linhas!")

Dica 2 — Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- Compreender os conceitos fundamentais da linguagem Python
 - Interpretar pequenos trechos de código
 - Criar programas simples utilizando:
 - variáveis,
 - operadores,
 - condicionais,
 - repetições,
 - listas e dicionários
 - Utilizar funções para organizar código
 - Reconhecer aplicações práticas do Python em diferentes contextos
-

Dica 3 — O que é Python?

Visão Geral da Linguagem

Características

- Linguagem de alto nível
- Sintaxe simples e legível
- Multiplataforma
- Código aberto

Áreas de aplicação

- Desenvolvimento web
- Automação
- Ciência de dados
- Inteligência Artificial
- Machine Learning

Por que aprender Python?

- Fácil para iniciantes
 - Muito utilizada no mercado
 - Grande comunidade e bibliotecas
-

Dica 4 — Ambiente de Desenvolvimento

Formas de executar Python

Modo interativo

Executa instruções linha a linha.
`print("Olá, Python!")`

Scripts Python

Arquivos com extensão `.py`
`# arquivo exemplo.py`
`print("Meu primeiro programa")`

Ferramentas comuns

- VS Code
 - PyCharm
 - Jupyter Notebook
 - geany
 - notepad++
-

Dica 5 — Variáveis e Tipos de Dados

Armazenando informações

```
nome = "Francisco Assis"
idade = 21
altura = 1.70
```

Tipos básicos

Tipo Exemplo

```
int    10
float  3.5
str    "Python"
bool   True
```

Verificando tipos

```
print(type(nome))
```

Dica 6 — Operadores

Operadores Matemáticos

```
a = 10
b = 2
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Operadores Relacionais

```
print(a > b)
print(a == b)
```

Dica 7 — Entrada e Saída de Dados

Interação com o usuário

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

Conversão de dados

```
idade = int(input("Digite sua idade: "))
```

Dica 8 — Estruturas Condicionais

Tomada de decisão

```
idade = 18
if idade >= 18:
    print("Maior de idade")
else:
    print("Menor de idade")
```

Estruturas utilizadas

- if
 - elif
 - else
-

Dica 9 — Exemplo de Condicional

Verificação de notas

```
nota = 6.5
if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

Observação: Atenção para a indentação.

Dica 10 — Estruturas de Repetição

Laço for

```
for i in range(5):
    print(i)
```

Laço while

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

Dica 11 — Listas

Coleções ordenadas

```
frutas = ["maçã", "bFranciscona", "uva"]
```

Operações básicas

```
print(frutas[0])
frutas.append("laranja")
```

Dica 12 — Percorrendo Coleções

Repetição com listas

```
nomes = ["Francisco", "Carlos", "Pedro"]
```

```
for nome in nomes:
    print(nome)
```

Dica 13 — Dicionários

Estrutura chave-valor

```
aluno = {
    "nome": "Andressa",
    "idade": 20
}
```

Acesso aos valores

```
print(aluno["nome"])
```

Dica 14 — Funções

Organização e reutilização

```
def saudacao(nome):
    print("Olá,", nome)
saudacao("Francisco")
```

Benefícios

- Reutilização
 - Organização
 - Facilidade de manutenção
-

Dica 15 — Exemplo Integrado (Desafio 1)

Pequeno programa em Python

```
def calcular_media(n1, n2):
    return (n1 + n2) / 2

nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
media = calcular_media(nota1, nota2)

print("Média:", media)
if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

Dica 16 — Aplicações do Python

Onde Python é utilizado?

Desenvolvimento

- Sites e APIs

Automação

- Rotinas administrativas
- Manipulação de arquivos

Ciência de Dados

- Análise de dados
- Visualização

Inteligência Artificial

- Machine Learning
 - Processamento de linguagem natural
-

Dica 17 — Relação com Data Science e IA

Python como linguagem estratégica

Bibliotecas populares

- Pandas
- NumPy
- Scikit-learn
- TensorFlow

Exemplos práticos

- Recomendação de filmes
 - Previsão de vendas
 - Chatbots
 - Análise de dados
-

Dica 18 — Exercício Prático

Desafio da Aula

Desenvolva um programa que:

1. Receba o nome do aluno
2. Receba duas notas
3. Calcule a média
4. Informe:
 - aprovado,
 - recuperação,
 - ou reprovado

Já solucionado em sala (Slides)

Desafio extra

Exibir:

- “Excelente desempenho” para média acima de 9

Dica 19 — Síntese da Aula

Conceitos estudados

- ✓ Introdução ao Python
- ✓ Variáveis e tipos
- ✓ Operadores
- ✓ Condicionais
- ✓ Repetições
- ✓ Listas e dicionários
- ✓ Funções
- ✓ Pequenos programas
- ✓ Aplicações em IA e Ciência de Dados

Dica 20 — Bibliografia

Referências

LAMBERT, Kenneth A.

Fundamentos de Python: estruturas de dados.

São Paulo: Cengage Learning Brasil, 2022.

NETTO, Amilcar; MACIEL, Francisco.

Python para Data Science e Machine Learning Descomplicado.

Rio de Janeiro: Alta Books, 2021.

YUXI, Hayden Liu.

Machine Learning com Python na prática.

São Paulo: Blucher, 2025.

Dica 21 — Encerramento

Perguntas e Discussão

Obrigado! 🚀

----- exemplo final O material abaixo, serve para ilustrar e aprofundar os conhecimentos em atividades do mundo Real.

Exemplo simples de Ciência de Dados e Machine Learning com Python

O exemplo abaixo:

- importa um arquivo .CSV,

- realiza análise básica dos dados,
- treina um modelo simples de Machine Learning,
- faz previsões.

O código foi pensado para iniciantes, alinhado às referências de Python, Data Science e Machine Learning da bibliografia.

Cenário do Exemplo

Vamos imaginar um arquivo chamado:
alunos.csv

Com os dados:

horas_estudo, frequencia, aprovado

2, 60, 0

5, 75, 1

1, 50, 0

8, 90, 1

6, 80, 1

3, 65, 0

7, 85, 1

Significado:

- horas_estudo → horas estudadas por semFrancisco
- frequencia → frequência nas aulas
- aprovado → 1 = aprovado / 0 = reprovado

Código Python Completo

```
# -----  
# Exemplo simples de Data Science + ML  
# -----  
# Importação das bibliotecas  
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score  
# -----  
# 1. Ler arquivo CSV  
# -----  
dados = pd.read_csv("alunos.csv")  
# Mostrar os dados  
print("Dados carregados:")  
print(dados)  
# -----  
# 2. Separar entradas e saída  
# -----  
# Variáveis de entrada  
X = dados[["horas_estudo", "frequencia"]]  
# Variável alvo  
y = dados["aprovado"]  
# -----  
# 3. Dividir dados de treino e teste  
# -----  
X_treino, X_teste, y_treino, y_teste = train_test_split(  
    X,  
    y,
```

```
test_size=0.3,
random_state=42
)
# -----
# 4. Criar modelo de Machine Learning
# -----
modelo = DecisionTreeClassifier()
# Treinar modelo
modelo.fit(X_treino, y_treino)
# -----
# 5. Fazer previsões
# -----
previsoes = modelo.predict(X_teste)
# -----
# 6. Avaliar desempenho
# -----
acuracia = accuracy_score(y_teste, previsoes)
print("\nPrevisões:")
print(previsoes)
print("\nAcurácia do modelo:")
print(acuracia)
# -----
# 7. Teste com novo aluno
# -----
novo_aluno = [[4, 70]]
resultado = modelo.predict(novo_aluno)
print("\nNovo aluno:")
if resultado[0] == 1:
    print("Aluno provavelmente APROVADO")
else:
    print("Aluno provavelmente REPROVADO")
```

Bibliotecas utilizadas

Pandas

Manipulação e leitura de dados.
import pandas as pd

Scikit-learn

Biblioteca de Machine Learning.
from sklearn.tree import DecisionTreeClassifier

Instalação das bibliotecas

pip install pandas scikit-learn

Conceitos trabalhados na atividade

Ciência de Dados

✓ Importação de CSV

- ✓ Manipulação de dados
- ✓ Separação de variáveis
- ✓ Análise básica

Machine Learning

- ✓ Treinamento de modelo
- ✓ Predição
- ✓ Avaliação de desempenho

Sugestões para evoluir o exemplo

Os alunos podem:

- adicionar mais colunas,
- testar outros algoritmos,
- gerar gráficos,
- usar dados reais,
- prever notas ao invés de aprovação.

Possível discussão em aula

Perguntas para os alunos

- Como o modelo “aprende”?
- O que influencia a aprovação?
- Mais dados melhorariam o modelo?
- O modelo pode errar? Por quê?

Versao _ Jupyter Notebook

Aula Prática — Introdução à Ciência de Dados e Machine Learning no Jupyter Notebook

Objetivo

Demonstrar, de forma introdutória, como utilizar Python no **Jupyter Notebook** para:

- importar dados .CSV,
- Franciscolisar informações,
- treinar um modelo simples de Machine Learning,
- realizar previsões.

Estrutura sugerida do Notebook

Célula 1 — Título (Markdown)

```
# Introdução à Ciência de Dados com Python
## Exemplo simples usando Jupyter Notebook
### Objetivos
- Ler arquivos CSV
- Manipular dados
- Criar um modelo de Machine Learning
- Fazer previsões
```

Célula 2 — Instalação das bibliotecas (opcional)

Executar apenas se necessário.

```
!pip install pandas scikit-learn
```

Célula 3 — Importação das bibliotecas

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Célula 4 — Criando os dados no próprio notebook

Ideal para evitar problemas iniciais com arquivos externos.

```
dados = {
    "horas_estudo": [2, 5, 1, 8, 6, 3, 7],
    "frequencia": [60, 75, 50, 90, 80, 65, 85],
    "aprovado": [0, 1, 0, 1, 1, 0, 1]
}
df = pd.DataFrame(dados)
df
```

Célula 5 — Salvando como CSV

```
df.to_csv("alunos.csv", index=False)
print("Arquivo CSV criado com sucesso!")
```

Célula 6 — Importando o arquivo CSV

```
dados_csv = pd.read_csv("alunos.csv")
dados_csv
```

Célula 7 — Explorando os dados

```
print("Informações gerais:")
print(dados_csv.info())
print("\nEstatísticas:")
print(dados_csv.describe())
```

Célula 8 — Separando entradas e saída

```
X = dados_csv[["horas_estudo", "frequencia"]]
y = dados_csv["aprovado"]
print("Entradas:")
print(X)
print("\nSaída:")
print(y)
```

Célula 9 — Divisão treino e teste

```
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)
print("Dados de treino:")
print(X_treino)
print("\nDados de teste:")
print(X_teste)
```

Célula 10 — Criando o modelo

```
modelo = DecisionTreeClassifier()
modelo.fit(X_treino, y_treino)
print("Modelo treinado com sucesso!")
```

Célula 11 — Fazendo previsões

```
previsoes = modelo.predict(X_teste)
print("Previsões:")
print(previsoes)
```

Célula 12 — Avaliando o modelo

```
acuracia = accuracy_score(y_teste, previsoes)
print("Acurácia do modelo:")
print(acuracia)
```

Célula 13 — Testando novo aluno

```
novo_aluno = [[4, 70]]
resultado = modelo.predict(novo_aluno)
if resultado[0] == 1:
    print("Aluno provavelmente APROVADO")
else:
    print("Aluno provavelmente REPROVADO")
```

Célula 14 — Conclusão (Markdown)

```
# Conclusão
Neste exemplo aprendemos:
- Importação de arquivos CSV
- Manipulação de dados com Pandas
- Separação de dados
- Treinamento de modelo
- Predição com Machine Learning
## Bibliotecas utilizadas
- Pandas
- Scikit-learn
```

Sugestões de expansão

Agora, espero que possam:

- adicionar mais registros,
- mudar os critérios de aprovação,
- usar gráficos,
- testar regressão linear,
- prever notas em vez de aprovação.

3. Motivacional e reflexão

Por Francisco Neto → A Arte de Enxergar Além: **A Jornada do Testador**

Muitas vezes, acreditamos que o sucesso está em confirmar que tudo vai bem. Mas Glenford J. Myers nos ensina uma lição mais profunda sobre a verdade: testar não é apenas validar o que já foi feito, é ter a coragem de olhar para a obra e perguntar: "**Onde você ainda pode falhar?**"

Enquanto o desenvolvimento é o ato sublime da construção, **o teste é o ato nobre da desconstrução**. Não é um ataque ao trabalho alheio, mas **um compromisso inabalável com a perfeição**. Ter a "psicologia do teste" é cultivar uma mentalidade que não se contenta com a superfície; **é entender que encontrar um erro não é um fracasso, mas uma vitória da transparência sobre a ilusão**.

Refleta: para que algo seja verdadeiramente resiliente, **ele precisa ser desafiado**. **O testador é aquele que protege o futuro**, garantindo que as falhas sejam reveladas na segurança do agora, para que não se tornem decepções amanhã.

Quebre os limites hoje, para que o seu sucesso seja inquebrável amanhã.

Comece a programar em Python3, hoje!

Falhas ocorrerão, mas apenas um estágio para alcançar a maturidade necessária.